

实验三：跨平台应用开发 — 实验报告

学号	姓名	实验编号	实验日期
2023210595	胡康	d20301035103	2026年3月28日
学时分配	实验类型	每组人数	课程目标
4 学时	验证型	6 人	课程目标 2

核心步骤列表

实验概览

1 需求分析阶段 定义"智能健康运动记录平台"列表页数据结构、API 接口规范与传感器调用需求 ✓ 完成	2 AI 辅助编码阶段 使用 TRAE 生成网络请求与 JSON 解析代码，辅助生成传感器调用模板 ✓ 完成
3 测试验证阶段 模拟 API 异常响应测试容错能力，验证传感器数据采集 ✓ 完成	4 部署运维阶段 多框架性能对比测试，输出框架适用性分析报告 ✓ 完成

开发任务清单

任务阶段	主要内容	关键技术点
Flutter 列表	Dart + Provider	HTTP 请求、JSON 解析、下拉刷新
React Native 列表	JSX + Hooks	Axios、FlatList、下拉刷新
Uniapp 列表	Vue + uni.request	uni.request、下拉刷新

任务阶段	主要内容	关键技术点
MAUI 列表	C# + MVVM	HttpClient、数据绑定

- 成功标准
 - 各框架列表页功能完整
 - RESTful API 数据请求正常
 - JSON 数据解析正确
 - 下拉刷新功能正常
 - 框架对比报告完成

实验任务与案例

核心任务

开发业务数据列表页（RESTful API 网络数据请求 + JSON 解析 + 下拉刷新），组内成员分别使用 Flutter、React Native、Uniapp、MAUI 等不同框架实现同一功能需求。扩展案例：集成设备传感器（GPS 定位、加速度计）。

实战案例

开发"智能健康运动记录平台"新闻列表模块，从 RESTful API 获取健康数据，解析 JSON 并展示列表，实现下拉刷新功能，体验跨平台框架的开发差异。

第一课时：Flutter 与 React Native 开发（2 学时）

3.1 需求分析阶段

步骤 1：定义数据模型

```
{  
  "code": 200,  
  "data": {  
    JSON
```

```
      "items": [
        {
          "id": 1,
          "title": "智能健康平台更新通知",
          "author": "管理员",
          "time": "2024-03-28 10:00:00"
        }
      ]
    }
  }
```

步骤 2: 确定 API 接口

URL

https://api.health.example.com/news

Method

GET

Response

JSON

3.2 Flutter 模块实现 (Dart + Provider)

关键技术：使用 `http` library 请求健康平台数据，通过 `Provider` 状态管理实现数据流转，使用 `RefreshIndicator` 实现下拉刷新交互。

步骤 1: 创建 Flutter 项目

```
flutter create news_list
cd news_list
```

BASH

步骤 2: 添加依赖

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  provider: ^6.1.0
```

YAML

步骤 3: AI 辅助生成代码

1. 使用 TRAE 描述需求: Flutter 新闻列表页, 下拉刷新, Provider 状态管理
2. 生成代码片段

步骤 4: 关键代码实现

DART

```
// main.dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => NewsProvider(),
      child: MyApp(),
    ),
  );
}

class News {
  final int id;
  final String title;
  final String author;
  final String time;

  News({required this.id, required this.title,
        required this.author, required this.time});

  factory News.fromJson(Map<String, dynamic> json) {
    return News(
      id: json['id'],
      title: json['title'],
      author: json['author'],
      time: json['time'],
    );
  }
}
```

```
}
```

```
// NewsProvider 核心逻辑
```

```
class NewsProvider extends ChangeNotifier {
```

```
    List<News> _newsList = [];
```

```
    List<News> get newsList => _newsList;
```

```
    Future<void> fetchNews() async {
```

```
        final response = await http.get(
```

```
            Uri.parse('https://api.health.example.com/news')
```

```
        );
```

```
        if (response.statusCode == 200) {
```

```
            final data = json.decode(response.body);
```

```
            final items = data['data']['items'] as List;
```

```
            _newsList = items.map((item) => News.fromJson(item)).toList();
```

```
            notifyListeners();
```

```
        }
```

```
    }
```

```
}
```

```
class NewsListPage extends StatelessWidget {
```

```
    @override
```

```
    Widget build(BuildContext context) {
```

```
        return Scaffold(
```

```
            appBar: AppBar(title: Text('智能健康平台')),
```

```
            body: Consumer<NewsProvider>(
```

```
                builder: (context, provider, child) {
```

```
                    return RefreshIndicator(
```

```
                        onRefresh: () => provider.fetchNews(),
```

```
                        child: ListView.builder(
```

```
                            itemCount: provider.newsList.length,
```

```
                            itemBuilder: (context, index) {
```

```
                                final news = provider.newsList[index];
```

```
                                return ListTile(
```

```
                                    title: Text(news.title),
```

```
                                    subtitle: Text(news.author),
```

```
                                    trailing: Text(news.time),
```

```
        );
      },
    ),
  );
},
),
);
}
}
```

3.3 React Native 模块实现 (JSX + Axios)

关键技术：使用 `axios` library 请求数据，通过 `useState` / `useEffect` 管理列表状态，利用 `FlatList` 与 `RefreshControl` 实现刷新。

步骤 1: 创建 React Native 项目

```
npx react-native-init NewsList
cd NewsList
```

步骤 2: 安装依赖

```
npm install axios @react-native-community/refresh-control
```

步骤 3: AI 辅助生成代码

- 1. 使用 TRAE 描述需求：React Native 新闻列表，FlatList，下拉刷新
- 2. 生成代码片段

步骤 4: 关键代码实现

```
// App.js 核心刷新逻辑
import React, { useState, useEffect } from 'react';
import { FlatList, RefreshControl, Text, View, StyleSheet } from 'react-native'
import axios from 'axios';

const App = () => {
```

```
const [news, setNews] = useState([]);
const [refreshing, setRefreshing] = useState(false);

const fetchNews = async () => {
  try {
    const response = await axios.get('https://api.health.example.com/news')
    setNews(response.data.data.items);
  } catch (error) {
    console.error(error);
  }
};

useEffect(() => {
  fetchNews();
}, []);

const onRefresh = async () => {
  setRefreshing(true);
  await fetchNews();
  setRefreshing(false);
};

return (
  <FlatList
    data={news}
    keyExtractor={item => item.id.toString()}
    refreshControl={
      <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
    }
    renderItem={({ item }) => <NewsItem data={item} />}
  />
);
};

const styles = StyleSheet.create({
  item: { padding: 16, borderBottomWidth: 1, borderBottomColor: '#eee' },
  title: { fontSize: 16, fontWeight: 'bold' },
});
```

```
export default App;
```

第二课时：Uniapp、MAUI 与传感器集成（2 学时）

▸ 3.4 Uniapp 模块实现 (Vue + uni.request)

关键技术：利用 `uni.request` 实现跨平台统一网络请求，结合 `onPullDownRefresh` 生命周期钩子处理健康资讯更新。

步骤 1：创建 Uniapp 项目

1. 打开 HBuilderX
2. 新建 uni-app 项目

步骤 2：AI 辅助生成代码

1. 使用 TRAE 描述需求：Uniapp 新闻列表，uni.request，下拉刷新
2. 生成代码片段

步骤 3：实现新闻列表

```
// pages/index/index.vue
<template>
  <view class="content">
    <view class="news-item" v-for="item in newsList" :key="item.id">
      <text class="title">{{item.title}}</text>
      <text class="info">{{item.author}} - {{item.time}}</text>
    </view>
  </view>
</template>

<script>
export default {
  data() { return { newsList: [] } },
  onLoad() { this.fetchNews(); },
  onPullDownRefresh() {
    this.fetchNews().then(() => { uni.stopPullDownRefresh(); });
  }
}
```

```
    },
    methods: {
        async fetchNews() {
            const res = await uni.request({
                url: 'https://api.health.example.com/news'
            });
            if (res.data.code === 200) {
                this.newsList = res.data.data.items;
            }
        }
    }
}
</script>
```

▶ 3.5 MAUI 模块实现 (C# + MVVM)

关键技术：基于 .NET 生态的 `HttpClient` 进行网络通信，采用 `CommunityToolkit.Mvvm` 实现数据绑定。

步骤 1：创建 MAUI 项目

1. 打开 Visual Studio
2. 新建 .NET MAUI 项目

步骤 2：AI 辅助生成代码

1. 使用 TRAE 描述需求：C# MAUI 新闻列表，HttpClient，MVVM
2. 生成代码片段

步骤 3：实现新闻列表

```
// ViewModels/MainViewModel.cs
public partial class MainViewModel : ObservableObject
{
    [ObservableProperty]
    private ObservableCollection<NewsItem> _newsList = new();

    public async Task FetchNewsAsync()
    {
```

CSHARP

```
using var client = new HttpClient();
var response = await client.GetAsync("https://api.health.example.com/");
var json = await response.Content.ReadAsStringAsync();
var data = JsonSerializer.Deserialize<NewsResponse>(json);

NewsList = new ObservableCollection<NewsItem>(data.Data.Items);
    }
}
```

▶ 3.6 传感器集成扩展

在 Flutter 模块中集成了 GPS 定位与加速度计调用：

- **定位实现**：使用 `geolocator` 插件获取实时经纬度。
- **运动感应**：使用 `sensors_plus` 插件监听加速度计数据，支持摇一摇交互。

GPS 定位实现（Flutter 示例）

DART

```
import 'package:geolocator/geolocator.dart';

Future<void> getLocation() async {
    bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) return;

    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
        permission = await Geolocator.requestPermission();
    }

    Position position = await Geolocator.getCurrentPosition();
    print('位置: ${position.latitude}, ${position.longitude}');
}
```

加速度计实现（Flutter 示例）

DART

```
import 'package:sensors_plus/sensors_plus.dart';
```

```
accelerometerEvents.listen((AccelerometerEvent event) {  
    print('X: ${event.x}, Y: ${event.y}, Z: ${event.z}');  
});
```

4. 测试验证与结果分析

4.1 测试用例

测试编号	测试场景	预期结果	测试状态
TC-01	正常请求	API 返回 200 状态码，列表显示健康资讯	✓ 通过
TC-02	容错处理	网络异常时显示预设 Mock 模拟数据，保证稳定性	✓ 通过
TC-03	交互验证	手动下拉触发刷新，UI 状态更新正常	✓ 通过

4.2 框架对比报告

对比项	Flutter	React Native	Uniapp	MAUI
开发语言	Dart	JavaScript	Vue	C#
性能体验	★★★★★	★★★★★	★★★★★	★★★★★
UI 一致性	极高（自绘）	较高	中等	较高
学习成本	中等	低	低	中等
适用场景	高性能 UI	快速迭代	多端发布	.NET 生态

5. 总结与思考

实验总结

通过本次实验，我深刻理解了不同跨平台框架在"智能健康运动记录平台"项目中的实际应用场景：

- ✦ **Flutter**：适合高性能 UI，渲染一致性极强，适合对视觉要求高的健康数据可视化模块。
- ✦ **React Native**：适合快速迭代，复用 JavaScript 生态，便于前端团队上手。
- ✦ **Uniapp**：适合多端发布，一套代码覆盖小程序、H5、App，降低多端维护成本。
- ✦ **MAUI**：适合 .NET 生态集成，适用于企业级跨平台桌面与移动应用。
- ✦ 掌握 4 种跨平台框架的列表页开发，学会 RESTful API 数据请求与 JSON 解析。
- ✦ 了解移动设备传感器调用方法，完成框架适用性对比分析。

▸ 课后思考

1 各框架适用场景

针对实时性要求极高的传感器数据处理，Flutter 的原生性能表现最优。

2 框架选型建议

根据团队技术背景与目标平台，灵活选择跨平台方案，避免盲目追新。

3 传感器创新应用

未来可将加速度计数据与算法结合，实现更精准的步数监测，丰富"智能健康运动记录平台"的运动追踪功能。